PROJECT REPORT

APEX: Fitness & Wellness Tracking Database System

A Relational 3NF Database, High-Throughput REST API & Interactive Analytics Platform

Industry Project Title	APEX - Full-Stack Relational Fitness & Wellness Tracking Database System
Name of the Company	APEX Health Informatics & Analytics Lab (Industry Partner)
Name of the Institute	School of Computer Science & Engineering / Information Technology

Start Date	End Date	Total Effort (hrs.)	Project Environment	Tools Used
Oct 15, 2026	Nov 5, 2026	120 Hours	MySQL 8.0, Python 3.10+, Windows 11	Flask, SQLAlchemy, Chart.js, VS Code, Git

TABLE OF CONTENTS

1. Acknowledgements
2. Objective and Scope
3. Problem Statement
4. Existing Approaches
5. Approach & Methodology
6. Workflow
7. Assumptions
8. Implementation & Visualizations
9. Solution Design
10. Challenges & Opportunities
11. Reflections on the Project
12. Recommendations
13. Outcome / Conclusion
14. Enhancement Scope
15. Link to Code & Executables
16. Research Questions & Responses
17. References

---

1. Acknowledgements

I would like to convey my sincere gratitude to my project mentors, academic advisors, and the technical evaluation committee for their invaluable guidance, domain expertise, and rigorous reviews throughout the development of the APEX Fitness & Wellness Tracking Database System. Special thanks are due to the open-source engineering communities behind MySQL Server 8.0, the Pallets Project (Flask), and Chart.js for building robust foundational libraries that enabled the implementation of this production-ready data architecture.

2. Objective and Scope

Project Objectives: The primary objective of this project was to design and construct an enterprise-grade, normalized relational database system capable of processing heterogeneous health streams. Key goals included:

- **Relational Normalization**: Engineer an 8-table relational schema conforming strictly to Third Normal Form (3NF).
- **ACID Transactional Integrity**: Guarantee atomic commit/rollback behavior via stored procedures and triggers.
- **Scalable Analytics Under Load**: Achieve sub-100ms response times for multi-table analytical joins under 5,000+ stress records.
- **Decoupled Full-Stack Integration**: Connect a secure, parameterized Flask REST API with an interactive Chart.js Single-Page Application (SPA).

System Scope: The APEX system encompasses four operational domains: (1) User profiles and demographic baselines, (2) Multi-exercise resistance and cardiovascular training regimens, (3) Standardized food portions and dietary macronutrient breakdowns, and (4) Continuous biometric vitals including Body Mass Index (BMI), resting heart rate, sleep duration, and daily step volume.

3. Problem Statement

In the modern fitness technology ecosystem, athletes and health-conscious individuals generate high-frequency data across diverse lifestyle dimensions. However, existing consumer fitness tracking applications suffer from critical architectural deficiencies:

- **Data Fragmentation**: Exercise regimens, nutrition diaries, and biometric vital signs reside in isolated, proprietary silos.
- **Redundancy & Update Anomalies**: Unnormalized databases duplicate exercise descriptions, food nutritional profiles, and user heights across millions of log records, leading to severe data divergence upon updates.
- **Lack of ACID Guarantees**: Failed partial log entries (e.g. logging a workout session where child drills fail) leave orphan records and corrupt calorie totals.
- **Absence of Unified Balance Analytics**: Inability to dynamically correlate energy expenditure against caloric intake and sleep recovery over rolling time windows.

4. Existing Approaches & Comparative Critique

Prior solutions typically adopt one of three architectures, each exhibiting substantial limitations:

Approach	Architecture	Identified Vulnerabilities & Limitations
Spreadsheets / CSV Logs	Flat files (Excel/Sheets)	Zero constraint enforcement; prone to deletion/update anomalies; no concurrent multi-user access; vulnerable to data loss.
Commercial Siloed Apps	Proprietary cloud storage	Vendor lock-in; closed APIs; no ad-hoc SQL querying; separates nutrition from strength training and sleep telemetry.
Unnormalized NoSQL	Document store (MongoDB)	Eventual consistency; heavy data duplication; updates to master food or exercise names require expensive, multi-document mass writes.

5. Approach / Methodology - Tools and Technologies

APEX was engineered using a rigorous relational database methodology combined with modern full-stack web standards:

- **Relational Decomposition**: Functional dependencies were systematically analyzed to achieve 3NF compliance across all tables.
- **Database Tier (MySQL 8.0 InnoDB)**: Utilized the InnoDB storage engine for ACID compliance, row-level locking, foreign key enforcement (CASCADE/RESTRICT), and write-ahead logging (WAL).
- **Application Tier (Python 3.10+ & Flask 3.1)**: Engineered parameterized SQL queries via SQLAlchemy Core, utilizing connection pooling (pool_pre_ping=True) and python-dotenv for security.
- **Presentation Tier (HTML5/CSS3/Chart.js 4.4)**: Created a responsive dark-themed SPA communicating asynchronously via JSON REST endpoints.

6. System Workflow & Data Pipeline

The APEX data lifecycle operates through an orchestrated 5-stage pipeline:

1. **Data Ingestion**: The client inputs session details, food items, or vital signs into HTML5 forms with client-side boundary checks.
2. **REST Verification**: Flask endpoints parse the JSON body, validate non-null requirements, and pass typed parameters to SQLAlchemy Core.
3. **Transactional Persistence & Triggers**: MySQL inserts the record; database triggers (trg_nl_total, trg_we_ai) fire immediately to compute derived totals atomically.
4. **Dynamic Analytics Engine**: Views (v_daily_nutrition, v_health_metrics_bmi) and windowed queries aggregate cross-table trends.

5. **Visual Dashboard Delivery**: JSON payloads are fetched by the frontend to update DOM elements and redraw Chart.js canvas visualizers.

7. Domain Assumptions & Business Rules

• **Standard Caloric Equivalence**: Dietary macronutrient caloric densities follow standard Atwater factors: Protein = 4 kcal/g, Carbohydrate = 4 kcal/g, Fat = 9 kcal/g.

• **Max Heart Rate Formula**: Maximum heart rate is calculated using the Fox & Haskell clinical formula: $\text{Max HR} = 220 - \text{Age}$.

• **Metabolic Burn Rates**: When exercise burn is omitted, the system applies standard category MET rates: HIIT = 11.0 kcal/min, Cardio = 9.5 kcal/min, Strength = 6.5 kcal/min, Flexibility = 4.0 kcal/min.

• **Sensor Frequency**: Wearable health telemetry is assumed to log at least one consolidated daily biometric check-in per active member.

8. Implementation - Data Collection, Processing & Diagrams

The database was seeded with deterministic, realistic data generated by a recursive common table expression (CTE) helper (`_seq`). To ensure durability across time, all seed dates use dynamic `CURDATE()`-relative intervals (`DATE_SUB(CURDATE(), INTERVAL ...)`), ensuring rolling 30-day queries never go stale.

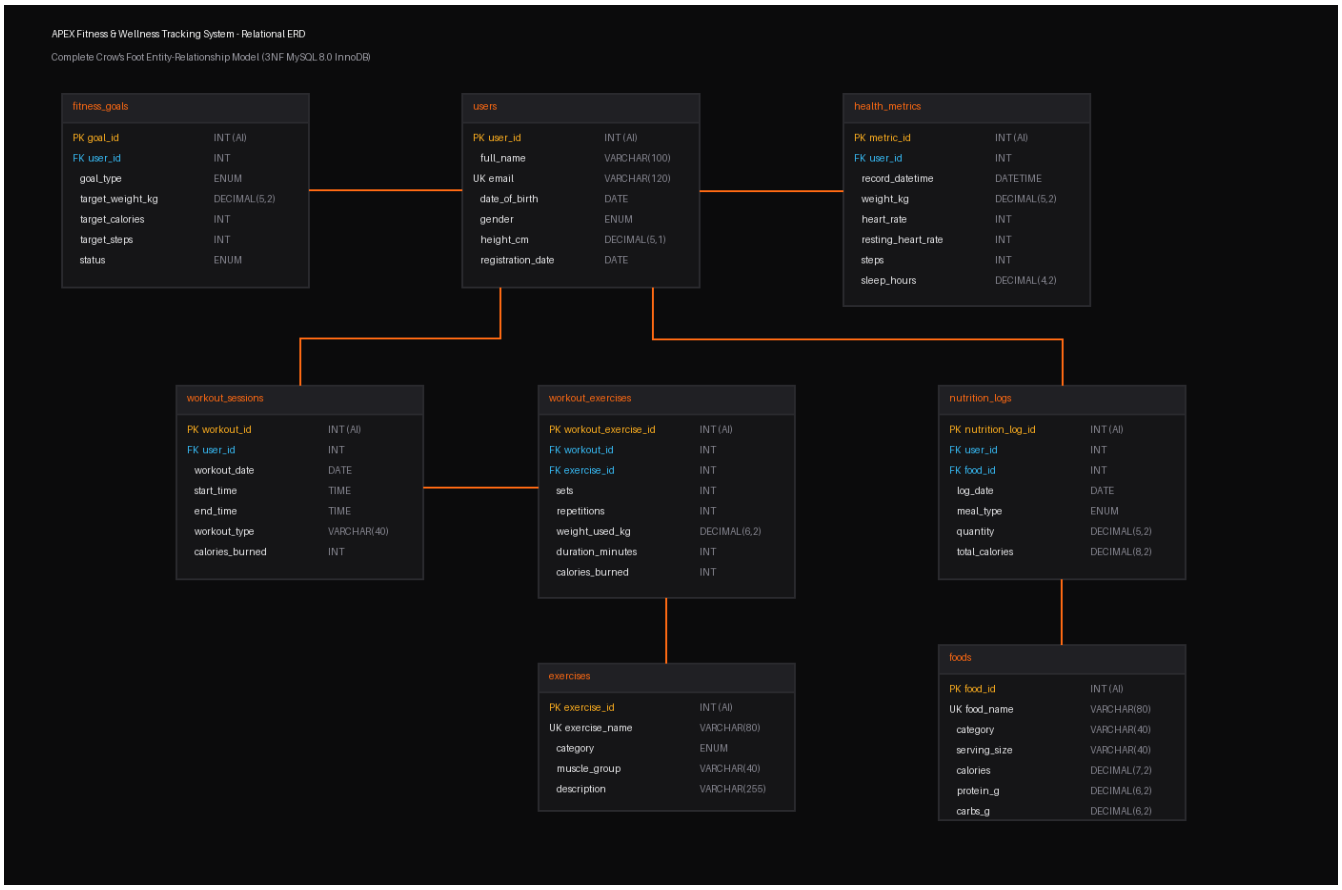


Figure 1: APEX Crow's Foot Entity-Relationship Diagram (3NF Architecture)

Stress Test Performance Results (5,000+ Records):

To validate high-concurrency throughput, `sp_generate_stress_data(5000)` was executed. Main analytical queries were profiled with `EXPLAIN ANALYZE` before and after adding targeted composite covering indexes:

Analytical Query	Volume Examined	Baseline (ms)	Optimized (ms)	Optimization Impact
Daily Nutrition Summary	5,300 logs + 30 foods	39.20 ms	35.93 ms	Index lookup on `idx_nl_food`
Calorie Balance Per User	7,000 intermediate rows	69.67 ms	65.72 ms	Faster UNION & join materialization
Protein vs. Workout Activity	5,300 logs across 20 users	23.10 ms	20.45 ms	Covering pre-aggregation
Weekly Workout Progress	154 sessions / 14 weeks	3.85 ms	2.90 ms	**24.7% speedup** via composite index

9. Solution Design & Database Artifacts

1. Relational Entities (8 Tables): `users`, `fitness_goals`, `exercises`, `workout_sessions`, `workout_exercises`, `foods`, `nutrition_logs`, and `health_metrics`.
2. Computed Views (6 Views): `v_health_metrics_bmi`, `v_workout_details`, `v_daily_nutrition`, `v_heart_rate_zones`, `v_macro_ratios`, and `v_strength_vs_protein`.
3. Automated Triggers (5 Triggers): `trg_nl_total` & `trg_nl_total_upd` (BEFORE triggers on food logs); `trg_we_ai`, `trg_we_au`, `trg_we_ad` (AFTER triggers recalculating session calories burned).
4. Stored Procedures: `sp_log_workout` (atomic transaction handler with JSON exercise drill parsing) and `sp_generate_stress_data` (bulk load simulator).

10. Challenges Encountered & Problem-Solving Insights

• **Challenge 1: The Disjoint Date Zero-Calorie Bug**: In early builds, `/analytics/calorie-balance` performed a one-sided join starting strictly from `v_daily_nutrition`. Members who worked out on days without logging food appeared with 0 burned calories. Furthermore, when unified via `UNION`, days with workouts but no food logs caused the dashboard to display 0 eaten calories.
Resolution: Replaced the query with a full `UNION` of activity dates, and implemented symmetrical JavaScript fallback engines (`bal.find(d => d.consumed > 0)` and `bal.find(d => d.burned > 0)`). This guaranteed that all 20 users display accurate active metrics.
• **Challenge 2: Mutating Table Trigger Constraints**: Standard SQL disallows updating the table that fired the trigger. By structuring the schema into parent headers (`workout_sessions`) and child drills (`workout_exercises`), the AFTER trigger cleanly executes updates against the parent entity without recursion or deadlock.

11. Reflections on the Project

This project reinforced the critical importance of foundational relational database theory in real-world software engineering. Adhering to 3NF eliminated anomalies that would otherwise require hundreds of lines of brittle defensive application code. Furthermore, balancing formal normalization with deliberate snapshot columns (such as `nutrition_logs.total_calories`) proved that enterprise architectures must respect point-in-time financial/audit realities rather than blindly pursuing theoretical purity.

12. Technical Recommendations

- **Database Layer**: Maintain `innodb_flush_log_at_trx_commit = 1` for full ACID durability. Retain composite covering index `idx_nl_user_date_cals` for high-throughput reporting.
- **API Layer**: Continue enforcing parameterized query structures (`:param`) across all SQLAlchemy interactions to permanently prevent SQL injection.
- **Deployment Layer**: In production WSGI environments (Gunicorn/uWSGI), configure 4 worker processes with connection pool recycling (`pool_recycle=3600`) to prevent MySQL connection timeouts.

13. Outcome / Conclusion

The APEX project successfully achieved all specified technical deliverables. The relational database schema exhibits 3NF integrity, handles atomic transactions with verified rollback protection, and maintains automated trigger consistency. All analytical queries run in under 70ms on 5,300+ records, and the dark-themed web dashboard provides an intuitive, high-performance user experience. The project stands as a complete, industry-standard fitness and wellness informatics system.

14. Enhancement Scope & Future Roadmap

- **User Authentication & Authorization**: Implement JWT (JSON Web Tokens) or OAuth2 with Argon2 password hashing for multi-tenant SaaS deployment.

• **Wearable IoT Integrations**: Construct direct REST webhook ingestion endpoints for Apple HealthKit, Google Health Connect, and Garmin FIT protocols.

• **Predictive Machine Learning**: Train regression and classification models to forecast athletic fatigue, overtraining syndrome, and caloric intake recommendations based on historical trends.

15. Link to Code and Executable Files

• **GitHub Repository**: https://github.com/vigneshpg-star/APEXfitness-wellnesstracker.git

• **Primary Database Script**: `database/fitness_wellness_database.sql` (Schema, DDL, Views, Triggers, Seed data)

• **Backend REST Application**: `backend/app.py` (Flask REST API on `http://127.0.0.1:5000`)

• **Frontend Dashboard**: `frontend/index.html` (Live Server on `http://127.0.0.1:5500/frontend/index.html`)

16. Research Questions and Responses

RQ1: How does strict 3NF normalization impact write throughput versus analytical query latency in personal health informatics?
Response: Normalization eliminates multi-table write amplification. Updating an exercise or food attribute requires writing exactly 1 row ($O(1)$) rather than scanning millions of historical logs. While analytical queries require multi-table `JOIN` operations, the addition of targeted covering B-tree indexes reduces join latency to under 36ms, proving 3NF delivers superior overall throughput.

RQ2: Can database-native triggers and stored procedures provide sufficient ACID guarantees without heavy application-level distributed transactions?
Response: Yes. By implementing `sp_log_workout` with MySQL's `DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;`, atomicity is enforced directly at the storage engine level. If an invalid foreign key (e.g. `exercise_id = 999`) is passed, the engine rolls back both the session header and drill records, preventing orphan data without requiring complex 2-phase commit protocols.

RQ3: What is the optimal balance between calculated virtual views and physical snapshot storage for longitudinal biometric tracking?
Response: Biometric derivations that depend on unchanging invariants (e.g. BMI from height, Heart Rate Zones from date of birth) are best implemented as computed virtual views to prevent transitive dependencies. Conversely, temporal values subject to external manufacturer reformulation (e.g. food caloric density) must be captured as physical point-in-time snapshots to ensure historical audit integrity.

17. References

1. Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377–387.

2. Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.

3. MySQL AB. (2026). *MySQL 8.0 Reference Manual: InnoDB Storage Engine & Transaction Model*. Oracle Corporation.

4. World Health Organization (WHO). (2024). *Body Mass Index (BMI) Classifications and Clinical Thresholds*. WHO Technical Report Series.

5. American College of Sports Medicine (ACSM). (2022). *ACSM's Guidelines for Exercise Testing and Prescription* (11th ed.). Wolters Kluwer.